Toward Efficient and Scalable Asynchronous Federated Learning via Stragglers Version Control

Chuyi Chen, Yanchao Zhao, Zhe Zhang, Wenzhong Li, *Member, IEEE*, and Jie Wu, *Fellow, IEEE*

***Abstract*—Asynchronous Federated Learning (AFL) has emerged as a promising paradigm to address the challenges posed by heterogeneous device environments in federated learning systems. However, the problem of low accuracy and slow convergence due to inconsistent updates from stragglers and normal clients remains severe in AFL. Previous works either discard or penalize the updates from stragglers, which can lead to the loss of valuable data or introduce bias into the model. Furthermore, existing AFL frameworks integrating synchronous optimization algorithms face the challenges of weak compatibility and scalability, limiting large-scale training. In this paper, we propose *DVAFL*, an efficient and scalable AFL framework that significantly improves the performance of AFL in terms of model accuracy and convergence speed by effectively utilizing and compensating for the updates from stragglers, while naturally integrating synchronous optimization algorithms. Specifically, DVAFL introduces a dynamic window protocol for adaptive aggregation to balance the contribution of stragglers and ensure faster and more stable convergence. Further, the version control mechanism corrects stale gradients by compensating for the missed model updates of stragglers, thereby improving model performance. Extensive experiments on three public datasets demonstrate that DVAFL achieves an average convergence speed 2.3× faster and an accuracy improvement of 5.5% compared to state-of-the-art AFL methods.**

***Index Terms*—Asynchronous federated learning, edge machine learning, system heterogeneity, version control mechanism, straggler.**

I. Introduction

FEDERATED Learning (FL) [1], [2] is a privacy-preserving distributed machine learning computing paradigm that has been widely adopted across various applications, including healthcare [3], mobile keyboard prediction [4], and IoT application [5]. In FL, edge devices (also called clients) perform local model training using their private data and share only the model parameters with the central server. The server aggregates these parameters from all participating clients to update the global model, thus preserving data privacy and enabling distributed computing services.

Nevertheless, clients in FL often comprise edge devices with diverse computational capabilities and network conditions, i.e., system heterogeneity [6], [7]. The server needs to wait for all clients to submit updates before aggregation, which will increase the latency and slow down the training process of the FL system. In such settings, the global model's training is often bottlenecked by the slowest clients, known as stragglers [8], which significantly hampers efficiency and scalability.

Asynchronous Federated Learning (AFL) [7], [8], [9], [10] is designed to alleviate the system heterogeneity problem. By allowing clients to upload updates independently of one another, AFL eliminates the need for synchronization and improves training efficiency. This computing paradigm can maximize the use of heterogeneous computing resources of each device while avoiding the overall system waiting problem caused by delays or disconnections of certain devices. However, this leads to another key problem, i.e., the inconsistency of model updates, where outdated updates from stragglers cause slower convergence and lower accuracy of the global model [11].

Existing approaches to managing stragglers in AFL focus on strategies such as penalizing, discarding, or preemptively uploading updates to mitigate their impact [8], [10], [13]. For instance, Xie et al. [8] penalize stragglers based on their staleness, SAFA [10] discards updates that are excessively outdated, and FedSA [13] requires stragglers to terminate local training early. Although these approaches reduce the negative impact of stragglers, they ignore the potential utility of the data held by stragglers and cannot effectively deployed to large-scale heterogeneous scenarios.

Differently, we argue that straggler-specific data and participation rates play a critical role in improving the global model performance and convergence speed. Discarding or down-weighting straggler updates may lead to the exclusion of valuable information. Our preliminary experiments (see Fig. 1) reveal that directly discarding and penalizing updates from stragglers leads to a significant decrease in model accuracy due to their data gaps and low participation rates, although it can improve training efficiency. Interestingly, using updates from stragglers improves model performance compared to dropping.

Received 30 December 2024; revised 23 July 2025; accepted 4 September 2025. Date of publication 12 September 2025; date of current version 9 January 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62172215 and in part by the A3 Foresight Program of NSFC under Grant 62061146002. Recommended for acceptance by Y. Deng. *(Corresponding author: Yanchao Zhao.)*

Chuyi Chen, Yanchao Zhao, and Zhe Zhang are with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: chuyichen@nuaa.edu.cn; yczhao@nuaa.edu.cn; zhezhang@nuaa.edu.cn).

Wenzhong Li is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China (e-mail: lwz@nju.edu.cn).

Jie Wu is with the China Telecom Cloud Computing Research Institute, Beijing 100033, China, and also with the Department of Computer and Information Sciences, Temple University, Philadelphia, PA 19122 USA (e-mail: jiewu@temple.edu).

Code available at https://github.com/NUAA-SmartSensing/DVAFL

Digital Object Identifier 10.1109/TMC.2025.3609568

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

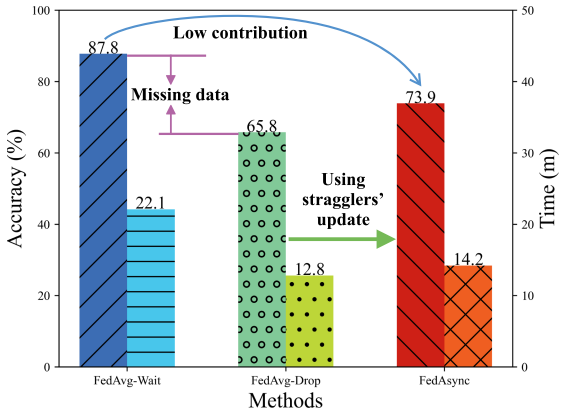


Fig. 1. Performance comparison of three different strategies for handling stragglers on FashionMNIST [12] dataset. Here, FedAvg-Wait is selectively waiting for updates from stragglers, FedAvg-Drop directly discards the straggler's update, and FedAsync uses lower weights to penalize stragglers.

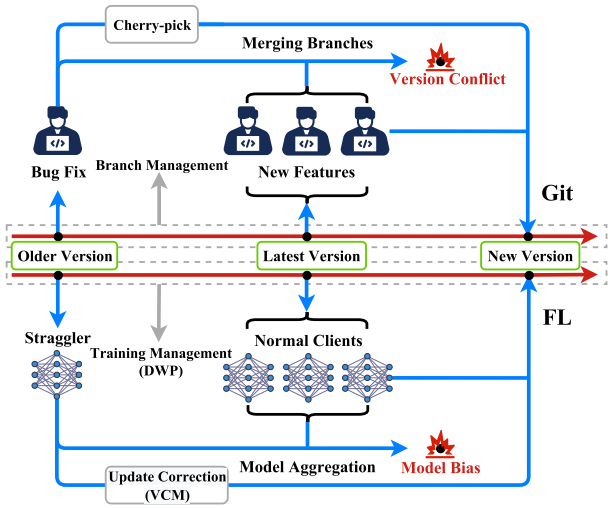


Fig. 2. Illustration of how version conflicts arise and are avoided in the AFL and Git processes. In Git, the Cherry-pick operation is used to apply bug fixes from the older version to the latest version, thereby avoiding conflicts. In DVAFL, model bias is mitigated by using VCM to align updates from stragglers with those of normal clients.

Thus, this implies to us *how to selectively retain outdated updates of stragglers and control the participation rate to achieve a balance between model performance and convergence speed.*

Drawing inspiration from software development, we integrated concepts from Git [14] into FL. As shown in Fig. 2, Git uses branch management to control the scope of each change. Similarly, we designed a Dynamic Window Protocol (DWP) for training to balance the contributions of different clients, ensuring faster and more stable convergence. Meanwhile, due to the presence of inherent stragglers, the aggregation of different versions of models can introduce model bias [1], [15], much like how merging different code versions in Git can lead to version conflicts. To address this, we borrowed Git's conflict resolution strategy to reduce model bias. Git uses cherry-pick to copy updates from older versions to the latest version, while we developed the Version Control Mechanism (VCM) to copy updates from stragglers in older models to the latest model, thereby aligning the updates of stragglers with those of normal clients.

Based on the above insights, we combine **D**WP and **V**CM to propose a novel **A**synchronous **F**ederated **L**earning framework, **DVAFL**, which utilizes adaptive dynamic aggregation windows and update corrections to accelerate model convergence and improve performance. Specifically, this framework consists of two key components. (1) DWP: Dynamically adjust the size of the aggregation window based on information of the expected completion time of clients. (2) VCM: Adopt gradient compensation to align the model updates of stragglers with those of normal clients. The main contributions of this paper are as follows:

- *An Efficient and Scalable AFL Framework via Managing the Updates of Stragglers:* We propose a novel AFL framework that efficiently integrates straggler updates and can be combined with other algorithms to enhance both the convergence speed and accuracy of AFL.
- *Adaptive Aggregation Window to Speed Up Model Convergence:* We introduce DWP, which adaptively manages the number of updates included in each aggregation round, thereby reducing the staleness effect of stragglers and speeding up model convergence in asynchronous systems.
- *Gradient Compensation for Stragglers to Improve Model Performance:* We propose VCM to compensate for the updates of outdated models held by stragglers, aiming to alleviate the conflict problem between model versions of different magnitudes and thus enhance overall model performance.
- *Theoretical Convergence Analysis:* We theoretically analyze the convergence of DVAFL for strongly convex and smooth problems, and show that it converges at the same level as FedAvg.
- *Enhanced Performance in Efficiency, Accuracy, and Scalability:* Our extensive experiments on three public datasets show that DVAFL achieves, on average, 2.3× faster convergence and 5.5% higher accuracy compared to state-of-the-art AFL methods, highlighting its effectiveness and scalability.

The rest of this paper is structured as follows. Section II provides a summary of related works relevant to FL. In Section III, we describe the AFL system model and present the problem formulation. In Section IV, we propose our framework, which consists of DWP and VCM. Afterward, we evaluate our proposed design both theoretically and experimentally in Sections V and VI, respectively. In Section VII, we conclude with a summary of future work and plans.

II. Related Work

A. System Heterogeneity in FL

FL faces significant challenges due to system heterogeneity among the participating devices [7], [16]. This heterogeneity is primarily reflected in the variations in computing power and communication bandwidth among devices, which differ in hardware (CPU, memory, battery) and network conditions (latency, bandwidth, packet loss rate).

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

Such disparities can lead to high latency in local training and communication between the server and clients, adversely affecting both the model's accuracy and overall efficiency [17]. To mitigate the impact of stragglers, many studies have proposed staleness-aware weighted aggregation techniques. For example, You et al. [18] suggested a temporally weighted aggregation method that adjusts updates based on client staleness, while Asyncfeded [19] quantified staleness by calculating the euclidean distance between a client's local model and the most recent global model.

Some approaches directly modify the client behavior. FedSAE [20] evaluates the reliability of individual devices by analyzing their complete training histories, enabling the server to adjust the workload for each client accordingly. This approach also prioritizes clients with better performance metrics derived from their training losses to optimize communication efficiency. TWAFL [21] introduced a two-way asynchronous FL algorithm that allows clients to transmit only part of their local model during certain rounds, while the full model is sent only intermittently. This reduces communication overhead and improves the system's overall efficiency. Additionally, grouping strategies have been employed to minimize communication costs. EAFL [7] clusters clients with similar characteristics for training, with intra-cluster communication conducted asynchronously and inter-cluster communication conducted synchronously. Zhang et al. [22] further advanced this approach by jointly considering both the optimization objectives and model update latency when grouping clients.

B. Asynchronous Federated Learning

AFL is a prominent strategy for expediting the training process in FL systems. In AFL, the server does not wait for all clients to upload their updates; instead, it allows clients to train on different versions of the global model independently. The first AFL algorithm, FedAsync, proposed by Xie et al. [8], enables clients to upload their updates at any time. However, aggregating individual local updates can cause fluctuations in model accuracy and reduce overall precision.

To address this issue, Chai et al. [9] proposed a method that clusters clients into performance tiers, applying synchronous learning within tiers and asynchronous learning across different tiers. Ma et al. [13] analyzed the convergence of FL, establishing a quantitative relationship between the frequency of aggregations and factors like training time and communication resources. Wu et al. [10] tackled the challenge of high-latency client updates by caching historical data and using a discriminative weighted aggregation algorithm to mitigate the impact of delayed client updates.

In contrast to previous methods, our proposed framework, DVAFL, predicts the expected completion time of client updates to synchronize client contributions effectively. Additionally, we leverage a Git-inspired system to integrate updates from stragglers, thereby mitigating their adverse effects on the global model's performance. This approach not only addresses the challenge of stragglers but also enhances the robustness and accuracy of the FL system.

TABLE I
MAIN NOTATIONS

Notation	Description
N	Total number of clients participating in FL
c_i	The specific client indexed by i
D_i	Dataset of client i
T	Global iterations
t_i	The number of rounds client i participates in FL
$F_i(w)$	local objective function of client i
w_i^t	The local model(update) of client i at round t
w^t	The global model at round t
γ_{agg}	The hyperparameter that controls the weight of the previous global model in the update process
η	Learning rate
p_i	The weights of client i
C^t	The set of clients selected at round t
U^t	The set of clients aggregated at round t
$\mathbb{C}^t$	The set of clients which are regarded as normal clients at round t
$\mathbb{S}^t$	The set of clients which are regarded as normal stragglers at round t
$\hat{\mathcal{S}}_i^t$	The estimated completion time of client i at round t
γ_{dwp}	The hyperparameter controlling the flexibility of the time window
T_{window}^t	Window size
Δg	The corrected bias used for VCM
ΔG	The version bias used for VCM
$\tilde{m}_i^{t_i}$	The corrected bias weighting factor of client i
M^t	The corrected bias weighting factor of global model
K	Maximum number of versions stored
γ_{vcm}	The hyperparameter that regulates the strength of VCM
λ	The hyperarameter for the moment estimation

III. PRELIMINARIES

FL [1] is a novel, privacy-preserving approach to distributed machine learning. In a typical FL scenario, there are two main entities: a central server and a set of clients, denoted as $\mathcal{C} = \{c_1, c_2, \ldots, c_N\}$. Each client c_i has access to its own local dataset D_i. For ease of reading, Table I provides a comparison of the meanings of the main notations. The aim of FL is to determine a global model w^* that minimizes the following objective function:

$$\min_w F(w) = \min_w \sum_{i=1}^{N} F_i(w), \tag{1}$$

where $F_i(w)$ represents the local objective function of client c_i.

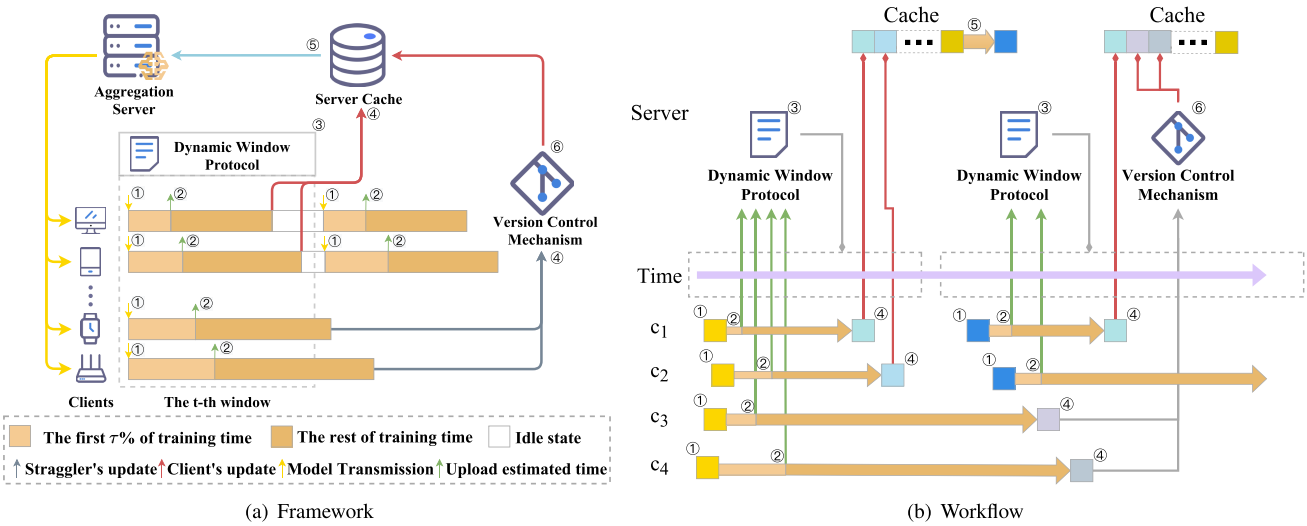
During the t-th communication round, the server selects a random subset of clients $\mathcal{C}^t \subseteq \mathcal{C}$, where the size of $\mathcal{C}^t$ is N^t, and distributes the current global model w^t to these clients. Upon receiving the global model w^t, each client $c_i \in \mathcal{C}^t$ updates the model using its local dataset D_i as follows:

$$w_i^t = w^t - \eta \nabla F_i(w^t), \tag{2}$$

where η is the learning rate. Clients that were not selected $\mathcal{C} - \mathcal{C}^t$ remain idle during this round. After gathering all the updates, the server aggregates the models from the selected clients $\{w_1^t, w_2^t, \ldots\}$ to update the global model w^{t+1}, using the following formula:

$$w^{t+1} = \sum_{i \in \mathcal{C}^t} p_i w_i^t, \tag{3}$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.



(a) Framework (b) Workflow

Fig. 3. Framework of DVAFL. (a) illustrates the structure of DVAFL. (b) outlines the workflow of DVAFL. The training process is as follows: ① The server broadcasts the global model to all clients; ② Clients send their estimated completion times to the server; ③ The DWP calculates and generates an appropriate time window for client updates; ④ Clients submit their locally trained models to the server; ⑤ The server aggregates the received local models into a new global model; ⑥ The VCM adjusts for any delayed updates from straggling clients, ensuring that their contributions are incorporated effectively.

where p_i denotes the weight of client c_i. In the FedAvg algorithm [1], each client's weight is proportional to the size of its local dataset, such that $p_i = \frac{|D_i|}{\sum_{i=1}^{N^t} |D_i|}$ where $|D_i|$ represents the number of data samples held by client c_i.

In the context of AFL, the server allows clients to train on different versions of the global model and does not wait for all clients to submit their updates. Instead, the server aggregates the models received from clients using the following function:

$$w^{t+1} = \gamma_{agg} * w^t + (1 - \gamma_{agg}) * \sum_{i \in U^t} \frac{p_i}{\sum_{j \in U^t} p_j} w_i, \quad (4)$$

where U^t represents the set of clients as defined by the FL method in use. Typically, U^t consists of clients whose updates the server has received between epochs $t-1$ and t. The update w_i from client i could have been generated at any point before epoch t. The parameter γ_{agg} determines the influence of the previous global model, where $\gamma_{agg} = 0$, results in the algorithm functioning as synchronous FL.

Various AFL approaches handle U^t differently. For instance, FedAsync [8] aggregates updates one at a time, which implies $U^t = \{c_i\}$. FedAt [9] groups clients into performance tiers, where the server synchronizes updates within each tier and aggregates asynchronously across tiers. Specifically, $U^t = \mathcal{C}_g^t$, where $C^t = \bigcup_{i=1}^{n_{tier}} C_i^t$ and n_{tier} is the number of tiers, with g is the tier that all clients in $\mathcal{C}_g^t$ being the tier where all clients have submitted their updates.

In this paper, the set of clients is defined as $U^t = \mathbb{C}^t \cup \mathbb{S}^t$, where $\mathbb{C}^t$ refers to the clients that submit updates within the expected time window, and $\mathbb{S}^t$ refers to stragglers who submit updates after being triggered by the server in the previous epoch and have now completed their training. The server dynamically adjusts the time window based on the estimated completion times of the clients.

IV. System Design

A. System Framework

As shown in Fig. 3, DVAFL consists of two core components:

- Dynamic Window Protocol (DWP): The DWP is a communication protocol that manages the timing of each training round. Clients estimate the time required to complete their local training based on their own conditions and upload this information to the server. The server uses these estimates to generate a dynamic time window, which governs the duration of each training round. This time window helps to manage concurrency and optimize training efficiency by determining how long the server waits before aggregating updates.
- Version Control Mechanism (VCM): The VCM integrates updates from stragglers with those from normal clients. This mechanism adjusts delayed updates from stragglers, ensuring that these updates do not negatively impact the accuracy of the global model. By incorporating these straggler updates effectively, the VCM helps improve the overall accuracy of the global model without discarding valuable information.

During each training round, clients upload their estimated training time according to the DWP. The server waits for all clients to complete their training within the time window defined by the DWP. Once training is finished, updates from normal clients are cached immediately, while updates from stragglers are processed through the VCM to ensure proper integration before being stored in the cache.

B. Dynamic Window Protocol

The DWP is a method designed to control the server's waiting time during each global round. Clients that submit their local

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Dynamic Window Protocol.

Input:

Server-Side:

Set of estimated completion times from all clients in round t, $\hat{\mathcal{S}}_i^t, i \in \mathcal{C}^t$.

Hyperparameter governing window redundancy, $\gamma_{dwp} \in [0, 1]$.

Client-Side:

Historical local training durations, $\{t_{\text{train},i}\}$.

Estimated network latency, $\hat{t}_{\text{network},i}$.

Output:

Server-Side:

Aggregation time window for round t, T^t_{window}.

Client-Side (for each client c_i):

Estimated completion time for round t, $\hat{S}_i^t$.

function Server-Side DWP$\{\hat{\mathcal{S}}_0^t, \hat{\mathcal{S}}_1^t, \ldots, \hat{\mathcal{S}}_i^t\}, \gamma_{dwp}$

  Update γ_{dwp} using (9) to control the flexibility of the time window

  Using clustering methods to cluster the clients based on their estimated completion time $\{\hat{\mathcal{S}}_i^t | c_i \in \mathcal{C}^t\}$ into two sets: $\mathbb{C}^t$ and $\mathbb{S}^t$ ▷ clustering methods can be K-means, DBSCAN, IQR, Z-score, or GMM.

  Sort $\{\hat{\mathcal{S}}_i^t | c_i \in \mathbb{C}^t\}$ to get the largest $\hat{\mathcal{S}}^t_{\max}$ in $\mathbb{C}^t$

  Sort $\{\hat{\mathcal{S}}_i^t | c_i \in \mathbb{S}^t\}$ to get the smallest $\hat{\mathcal{S}}^t_{\min}$ in $\mathbb{S}^t$

  Set the time window size $T^t_{window} = \hat{\mathcal{S}}^t_{\min} * \gamma_{dwp} + \hat{\mathcal{S}}^t_{\max} * (1 - \gamma_{dwp})$

  return T^t_{window}

end Function

function Client-Side DWP$\{t_{train,i}\}, \hat{t}_{network}$

  Estimate the shape parameter $\hat{\alpha}_i$ of the Pareto distribution for local training duration using (6).

  Compute the expected local training duration $\hat{t}_{\text{train},i}$ using (7).

  Calculate the total estimated completion time: $\hat{S}_i^t \leftarrow \hat{t}_{\text{train},i} + \hat{t}_{\text{network},i}$.

  return $\hat{S}_i^t$

end Function

updates within the specified time window are categorized as normal clients, while those submitting after the time window are considered stragglers. We define the set of normal clients in round t as $\mathbb{C}^t$, the set of stragglers as $\mathbb{S}^t$, and $\mathcal{C}^t = \mathbb{C}^t \cup \mathbb{S}^t$ represents all clients.

To distinguish between normal clients and stragglers, the server collects latency reports $\mathcal{S}^t = \{\mathcal{S}_0^t, \mathcal{S}_1^t, \ldots, \mathcal{S}_i^t\}$ from all clients. Inspired by insights from [23], [24], the DWP is specifically designed around the understanding that: (1) The dynamic variability of device hardware resources contributes to an increased uncertainty in training durations. (2) Training times exhibit significant variability due to sporadic resource contention and hardware performance fluctuations, occasionally leading to extreme durations and thus forming a heavy-tailed distribution [25].

Under the DWP, each client reports an estimated completion time, denoted as S_i^t, to the server upon reaching a locally defined training threshold τ. This threshold is typically set at 10% of the total training process duration. The estimate integrates both the expected training duration $\hat{t}_{train}$ and the communication delay $\hat{t}_{network}$. Throughout the training process, clients record the duration of each training batch, denoted as $t_{train,b}$, where b represents the batch index. Upon hitting the DWP threshold, clients use the collected batch times $\{t_{train,b}\}$ to compute the estimated overall training time $\hat{t}_{train}$. Given the heavy-tailed nature of training durations, we use the Pareto distribution to model the training time. Its probability density function is given by:

$$f(x; x_m, \alpha) = \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, \quad x \geq x_m > 0, \quad \alpha > 0, \tag{5}$$

where x_m is the minimum threshold and α is the shape parameter (tail index).

Suppose the client has observed a sample set of the first 10% training durations $t_{train,1}, t_{train,2}, \ldots, t_{train,n}$. the shape parameter α of the Pareto distribution is estimated using the Maximum Likelihood Estimation (MLE) method. The estimator for α is given by:

$$\hat{\alpha} = \frac{n}{\sum_{i=1}^n \ln \frac{t_{train,i}}{t_{\min}}}, \tag{6}$$

where $t_{\min} = \min\{t_{train,i}\}$.

A key property of the Pareto distribution is that the expected value of the random variable, conditioned on it exceeding a certain duration $t_{observed}$, can be analytically derived. This allows for the prediction of the residual duration. Specifically, the expected total training time is given by the conditional expectation:

$$E[t_{train} | t_{train} > t_{observed}] = \frac{\alpha}{\alpha - 1} \cdot t_{observed}. \tag{7}$$

The communication time for each client is estimated based on the size of the local model update and the available network bandwidth. After gathering all client estimates $\hat{\mathcal{S}}^t = \{\hat{\mathcal{S}}_0^t, \hat{\mathcal{S}}_1^t, \ldots, \hat{\mathcal{S}}_i^t\}$, the server performs a classification to partition the clients into two distinct groups: normal clients $\mathbb{C}^t$ and stragglers $\mathbb{S}^t$. To this end, the DWP can employ one of five clustering algorithms: K-means [26], DBSCAN [27], IQR [28], Z-score [29], or GMM [30]. Following this classification, the server determines the duration of the time window for the current round, T^t_{window}. This window is calculated as a weighted average of the boundary completion times of the two client groups:

$$T^t_{window} = \gamma_{dwp} \cdot \hat{\mathcal{S}}^t_{\min} + (1 - \gamma_{dwp}) \cdot \hat{\mathcal{S}}^t_{\max}, \tag{8}$$

where $\hat{\mathcal{S}}^t_{\max} = \max_{i \in \mathbb{C}^t} \hat{\mathcal{S}}_i^t$ represents the maximum estimated completion time among the normal clients, $\hat{\mathcal{S}}^t_{\min} = \min_{i \in \mathbb{S}^t} \hat{\mathcal{S}}_i^t$ corresponds to the minimum estimated completion time among the stragglers, and γ_{dwp} is a self-adaptive hyperparameter that governs the adaptability of the time window. A higher value of γ_{dwp} yields a more lenient time window, accommodating a greater number of potential stragglers, whereas a lower value

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

results in a stricter time window that prioritizes the expedited submissions of normal clients.

To enhance the adaptability of the aggregation time window, we incorporates a dynamic adjustment mechanism analogous to the congestion control principles of the Transmission Control Protocol (TCP). In a manner similar to how TCP modulates its transmission window in response to network feedback, the server in our protocol continuously monitors the empirical arrival times of client updates to inform the adjustment of the time window for subsequent training rounds. As detailed in Algorithm 1, the server adjusts the parameter γ_{dwp} using the following formula:

$$\gamma_{dwp} = \frac{1}{2} \cdot \gamma_{dwp} + \frac{\mathcal{S} - \hat{\mathcal{S}}^t_{\min}}{\hat{\mathcal{S}}^t_{\min} - \hat{\mathcal{S}}^t_{\max}}, \tag{9}$$

where $\mathcal{S}$ denotes the actual arrival time of the last client that successfully submitted its update within the established time window T^t_{window}. Any client initially classified as "normal" that fails to meet this deadline is subsequently reclassified as a straggler for that round. This dynamic adjustment mechanism ensures that the time window remains responsive to the observed performance of the client cohort, thereby optimizing the aggregation process and enhancing overall system efficiency.

C. Version Control Mechanism

Algorithm 2: Version Control Mechanism.

Input: The client's update w_i, time stamp t'_i, client's weight p^t_i

Output: The corrected update $\tilde{w^t_i}$ and $\tilde{p^t_i}$

1: **if** $t'_i \neq t$ **then**▷ w^t_i is the update from straggler
2: **if** $t - t'_i > K$ **then**▷ Straggler's update is too outdated
3: discard w^t_i
4: **end if**
5: $\tilde{w^t_i} = Update_Correction_Method(w^t_i)$. ▷ Using (12)
6: $\tilde{p^t_i} = Weight_Correction_Method(p^t_i)$. ▷ Using (10)
7: **else**
8: **return**w_i, p^t_i ▷updates from normal clients are directly stored in the cache
9: **end if**
10: **return** $\tilde{w^t_i}, \tilde{p^t_i}$

In many asynchronous algorithms, stragglers are defined as clients with significant latency, but this definition is somewhat vague. Therefore, we first differentiate stragglers from normal clients. By integrating this concept with DWP, we define stragglers as clients submitting updates based not on the current global model w^t but on an earlier global model w^{t-k_i}, where $0 < k_i < t$. The key distinction is that stragglers use outdated models, making their updates incompatible with the current global model.

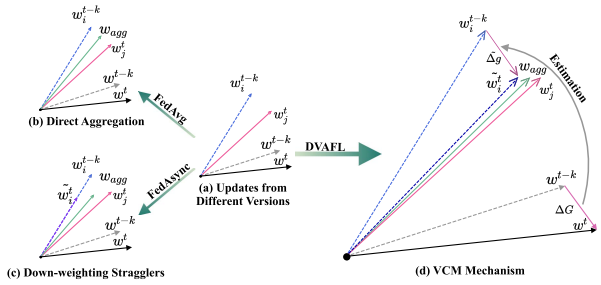


Fig. 4. Illustration of the update correction method in the VCM.

We designed VCM to address the issue of update mismatches by detecting, correcting, and aligning the updates uploaded by clients. VCM stores multiple global models $w^{t-K}, w^{t-K+1}, \ldots, w^t$, with K being a hyperparameter limited by storage capacity. Each global model represents a different version. When VCM receives an update $w^{t-k_i}_i$, as outlined in Algorithm 2, it identifies the source of the update. If the update comes from a normal client, VCM immediately caches it for later aggregation. If it comes from a straggler, VCM first assesses its staleness; if the update is excessively outdated ($k_i > K$), VCM discards it. Otherwise, VCM adjusts the update's weights and values to minimize conflicts during aggregation. Once the correction and alignment are completed, VCM stores the result in the cache. It is important to note that discarding an update does not affect the client's participation rate, as VCM is solely responsible for processing the uploaded updates.

Next, we will describe the specific implementations of weight correction and update correction in VCM. We adopt the weighting function from FedAsync [8] as the weight correction method. It typically uses the following formula for correction:

$$\tilde{p_i} = s(k_i) \times p_i, \tag{10}$$

where $s(k_i)$ is a staleness function. Several forms of $s(k_i)$ can be applied:

- Constant: $s(k_i) = 1$.
- Polynomial: $s_a(k_i) = (k_i + 1)^{-a}$.
- Hinge: $s_{a,b}(k_i) = \begin{cases} 1, & \text{if } k_i \leq b \\ \frac{1}{a(k_i - b) + 1}, & \text{otherwise} \end{cases}$.

Previous study has shown that merging updates trained on different versions of the global model negatively impacts accuracy [15]. As illustrated in Fig. 4, direct aggregation or down-weighting methods fail to effectively align updates from stragglers and normal clients. To address this, VCM first calculates the version bias ΔG, defined as

$$\Delta G = w^t - w^{t-k_i}. \tag{11}$$

Based on ΔG, a corrected bias $\tilde{\Delta} g$ is computed using methods described subsequently. The straggler's update is then adjusted as follows:

$$\tilde{w^t_i} = w^{t-k_i}_i + \gamma_{vcm} \tilde{\Delta} g, \tag{12}$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

where γ_{vcm} is a hyperparameter that controls the correction intensity. It is crucial to emphasize that VCM estimates the deviation of the straggler's local model by leveraging the delta between different global model versions. This approach contrasts with the direct weighted averaging of the straggler's updates and the global model parameters. The latter constitutes the foundational principle of down-weighting schemes, such as those adopted by FedAsync, from which our proposed methodology fundamentally diverges.

To estimate the corrected bias $\tilde{\Delta g}$, we introduce the notation $\hat{m}_i^{t_i}$ and $\hat{M}^t$ to represent the calibrated weights of the local and global models, respectively. The corrected bias is then given by

$$\tilde{\Delta g} = \frac{\hat{m}_i^{t_i}}{\hat{M}^t} \Delta G, \tag{13}$$

where t_i denotes the number of rounds client i participates in FL, distinguishing it from the global iteration t.

Two distinct methodologies have been designed to calculate the weights $\hat{m}_i^t$ and $\hat{M}^t$. The first and simplest method is to directly use the version bias ΔG as the corrected bias $\tilde{\Delta g}$, which was proposed in our previous work [31], as follows:

$$\frac{\hat{m}_i^{t_i}}{\hat{M}^t} = 1. \tag{14}$$

Though this method is straightforward, direct use of the version bias $\tilde{\Delta g}$ with a fine-tuned hyperparameter γ_{vcm} can achieve a good performance. Moreover, it is easy to implement and has a low computational cost. In our experiments, we have successfully validated the feasibility of this approach.

The second method is inspired by the Adam optimizer [32], which utilizes the exponential moving average of updates as the weight for the bias. We introduce the exponential moving average of the updates as the weighting factor $\hat{m}^t$ and $\hat{M}^t$. Two weight calculation formulas have been designed, which use first-moment estimation and second-moment estimation, respectively, to estimate the exponential moving average and thus serve as weights for the version bias and the corrected bias, respectively.

- The first-moment estimation is calculated as follows:

$$m_i^{t_i} = \lambda m_i^{t_i-1} + (1-\lambda)\|g_i^{t-k_i}\|, \tag{15}$$

$$M^t = \lambda M^{t-1} + (1-\lambda)\|G^t\|, \tag{16}$$

  where λ is a hyperparameter, $g_i^{t-k_i} = w_i^{t-k_i} - w_i^{t-k}$ and $G^t = w^t - w^{t-1}$.
- The second-moment estimation is calculated as follows:

$$m_i^{t_i} = \lambda m_i^{t_i-1} + (1-\lambda)\|g_i^{t-k_i}\|^2, \tag{17}$$

$$M^t = \lambda M^{t-1} + (1-\lambda)\|G^t\|^2. \tag{18}$$

When a new client joins the FL training process, the VCM mechanism initializes the client's local weights using the global weight coefficient, M^t. As both $m_i^{t_i}$ and M^t are initialized to zero, they exhibit a bias towards zero in the initial stages of training. Consequently, a bias-correction step is necessary for both $m_i^{t_i}$ and M^t to mitigate this effect, a technique adapted from the Adam optimization algorithm [32]. To simplify the calculation, we let $\mathbb{M} = m_i^{t_i}$ or M^t. For example, when $\mathbb{M} = m_i^{t_i}$, we use the following formula to correct the $m_i^{t_i}$:

$$\hat{m}^{t_i} = \frac{\mathbb{M}}{1-\lambda^{t_i}}. \tag{19}$$

Algorithm 3: Workflow of DVAFL.

Input: The number of iterations T, the number of local iterations E, hyperparameter τ
Output: The final global model w^t

```
Run Server Process on the server and Client Process on each client in parallel
Process Server():
Initialize the global model w^0
for t = 0 to T do
    Randomly select a subset of clients C^t ⊆ C
    Send the current global model w^t to clients in C^t
    Execute DWP to obtain the time window size T^t_window
    loop wait for T^t_window seconds to receive updates from clients
        Receive updates w_i from clients
        Execute VCM to obtain the corrected updates w̃_i
        Cache the updates w^t_i
    end loop
    Aggregate the updates in the cache to generate the new global model w^{t+1} by (3)
end for
Process Client():
if selected by the server then
    Receive the current global model w^t
    Initialize b = 0
    for local iteration i = 0 to E do
        Update local model by (2)
        if reaching the τ% entire training process then
            Execute DWP to obtain the estimated completion time Ŝ^t_i
            Upload Ŝ^t_i to the server
        end if
    end for
    Upload local update w^t_i to the server
end if
```

D. Workflow of DVAFL

In this subsection, we provide a detailed overview of the DVAFL workflow. As illustrated in Fig. 3 and Algorithm 3, in each communication round (denoted as the t-th round), the server randomly selects a set of available clients C^t to participate in local training. The server starts by sending the latest global model w^t to the selected clients (①). Upon receiving the global model, each client (denoted as client c_i) sets up its local training process. During training, each client records the time taken for each batch $t_{train,b}$. After completing $\tau\%$ of the training, the client follows the DWP (detailed in Algorithm 1) to estimate its completion time $\hat{S}_i^t$ and uploads this estimate to the server (②).

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

After completing local training, the client uploads its model update w_i^t to the server and enters an idle state, awaiting further selection (④ and ⑤).

On the server side, after receiving all estimated completion times $\hat{\mathcal{S}}^t$, the server applies the DWP (detailed in Algorithm 1) to set the time window for the current round $T_{windows}^t$ (③). During this window, the server collects updates from the clients. If an update w_i comes from a straggler, it is processed through the VCM (detailed in Algorithm 2), which adjusts it to align with the current global model as w_i^t (⑥). All updates, both from normal clients and stragglers, are then cached (④ and ⑥). Once the time window closes, the server retrieves all cached updates and uses an aggregation algorithm to compute the new global model w^{t+1}, following (3) (⑤). Following this, a new communication round begins.

Furthermore, it is worth noting that, from the aggregation algorithm's viewpoint, the process is synchronous, with no visible version inconsistencies in the updates. This makes the DVAFL framework non-intrusive, allowing seamless integration with any aggregation algorithm and compatibility with synchronous FL methods.

V. Convergence Analysis

In this section, we provide a detailed convergence analysis of DVAFL. To begin, we outline the following assumptions for the objective function $f_i(w)$:

Assumption 1 (Lipschitz Continuity): The objective function $f_i(w)$ is Lipschitz continuous with constant $L_i > 0$. This ensures that $\forall w_1, w_2 \in \mathbb{R}^d$, the gradients of F_i satisfy the condition: $\|\nabla F_i(w_1) - \nabla F_i(w_2)\|_2 \leq L_i \|w_1 - w_2\|_2$.

Assumption 2 (Bounded Gradient): The gradient of the objective function $F_i(w)$ is bounded, implying that $\forall w \in \mathbb{R}^d$: $\|\nabla F_i(w)\|_2 \leq B$.

Assumption 3 (μ-Strongly Convex): The objective function $f_i(w)$ is μ-strongly convex. This ensures that $\forall w_1, w_2 \in \mathbb{R}^d$: $F_i(w_1) \geq F_i(w_2) + \nabla F_i(w_2)^T (w_1 - w_2) + \frac{\mu}{2}\|w_1 - w_2\|_2^2$.

These assumptions are standard in FL convergence analysis [33], [34], [35]. Next, we analyze the convergence of DVAFL under these assumptions.

Lemma 1: When γ_{vcm} is optimally chosen, the weight ratio between the local and global models in the VCM is bounded by:

$$\gamma_{vcm} \frac{\hat{m}^t}{\hat{M}^t} \leq 1, \tag{20}$$

Proof: The following cases are discussed in detail below.

- *Case 1:* According to the (14), the Lemma 1 holds.
- *Case 2:* According to the (15) and (16), we have:

$$\frac{\hat{m}^t}{\hat{M}^t} = \frac{\frac{\sum_{i=1}^{t_c} \lambda^{t_c - i} \|g_c^i\|}{1-\lambda^{t_c}}}{\frac{\sum_{i=1}^{t} \lambda^{t-i} \|G^i\|}{1-\lambda^t}}. \tag{21}$$

According to Assumption 2, we have:

$$\frac{\hat{m}^t}{\hat{M}^t} \leq \frac{B}{\frac{\sum_{i=1}^{t} \lambda^{t-i} \|G^i\|}{1-\lambda^t}} = \Upsilon, \tag{22}$$

where Υ is a constant, $\Upsilon = \frac{1-\lambda^t}{\sum_{i=1}^{t} \lambda^{t-i} v_i}$. When $\gamma_{vcm} \leq \frac{1}{\Upsilon}$, the Lemma 1 holds.

- *Case 3:* This case is similar to Case 2. We can get the similar result as Case 2. When $\gamma_{vcm} \leq \frac{1}{\Upsilon'}$, $\Upsilon' = \frac{1-\lambda^t}{\sum_{i=1}^{t} \lambda^{t-i} v_i^2}$, the Lemma 1 holds.

□

Theorem 1: Suppose Assumptions 1, 2, and 3 hold. Then, the proposed DVAFL algorithm satisfies the following convergence bound:

$$\mathbb{E}|w^t - w^*|^2 \leq D\rho^t + \frac{C}{\eta\mu\alpha}, \tag{23}$$

where $D = \Phi^0$ and $\rho = 1 - \eta\mu\alpha$. The constant C is given by

$$C = \left((1-\alpha)K + 1 + \frac{KL}{\mu}\right)\eta^2 E^2 Q^2 + 2\eta^2 KL(1-\alpha)\Gamma_{\mathbb{S}} - 2\eta\alpha\Gamma_{\mathbb{C}}, \tag{24}$$

where α is the proportion of normal clients, K is the maximum staleness, L is the Lipschitz constant, μ is the strong convexity parameter, E is the number of local epochs, Q bounds the gradient, and $\Gamma_{\mathbb{S}}$, $\Gamma_{\mathbb{C}}$ denote the optimality gaps for stragglers and normal clients, respectively.

Proof:

$$\|w^{t+1} - w^*\|^2 = \|\sum_{i\in\mathbb{C}^t} p_i(w^t - \eta g_i^t) + \sum_{i\in\mathbb{S}^t} p_i(w^{t-k_i} - \eta g_i^t) + \sum_{i\in\mathbb{S}^t} p_i\left(\gamma_{vcm}\frac{\hat{m}^t}{\hat{M}^t}(w^t - w^{t-k_i})\right) - w^*\|^2. \tag{25}$$

To simplify the proof, we define $\bar{G}$ as follows:

$$\bar{G} = \sum_{i\in\mathbb{C}^t} p_i g_i^t + \sum_{i\in\mathbb{S}^t} p_i g_i^{t-k_i}. \tag{26}$$

Then by the Lemma 1, we have:

$$\begin{aligned}
&\|w^{t+1} - w^*\|^2 \\
&\leq \|\sum_{i\in\mathbb{C}^t} p_i(w^t - \eta g_i^t) \\
&\quad + \sum_{i\in\mathbb{S}^t} p_i(w^{t-k_i} - \eta g_i^t + w^t - w^{t-k_i}) - w^*\|^2 \\
&= \|w^t - w^*\|^2 + \underbrace{\eta^2\|\bar{G}\|^2}_{B_1} - \underbrace{2\eta\langle w^t - w^*, \bar{G}\rangle}_{B_2}.
\end{aligned} \tag{27}$$

First, according to the Assumption 2, we can bound the local updates $\|g_i^t\|^2$ as follows:

$$\|g_i^t\|^2 = \|\sum_{k=0}^{E-1} \nabla F_i(w_i^{t,k}, \xi_i^k)\|^2 \leq E^2 Q^2, \tag{28}$$

where $w_i^{t,0} = w^t, w_i^{t,E} = w_i^t$ and ξ_i^k is the sample used in the k-th iteration.

Next, we will analyze B_1 and B_2 respectively.

$$B_1 \leq \eta^2\left(\sum_{i\in\mathbb{C}^t} p_i\|\nabla F_i(w^t)\|^2 + \sum_{i\in\mathbb{S}^t} p_i\|\nabla F_i(w^{t-k_i})\|^2\right)$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

$$\leq \eta^2 E^2 Q^2, \tag{29}$$

where the first inequality is due to the convexity of $\|\cdot\|^2$ and the second inequality is due to the Assumption 2.

By the Cauchy-Schwarz inequality, we have [34]:

$$-2\langle w^t - w^*, \nabla F_i(w^t)\rangle \leq \frac{1}{\eta}\|w^t - w^*\|^2 + \eta\|\nabla F_i(w^t)\|^2. \tag{30}$$

By the Assumption 3, we have [34]:

$$-\langle w^t - w^*, \nabla F_i(w^t)\rangle \leq F_i(w^*) - F_i(w^t) - \frac{\mu}{2}\|w^t - w^*\|^2. \tag{31}$$

Applying (30) and (31) to B_2, we obtain the following results:

$$\begin{aligned} B_2 = & -2\eta\left(\sum_{i\in\mathbb{C}^t} p_i\langle w^t - w^*, \nabla F_i(w^t)\rangle \right. \\ & \left. + \sum_{i\in\mathbb{S}^t} p_i\langle w^t - w^*, \nabla F_i(w^{t-k_i})\rangle\right) \\ \leq & -2\eta\left(\sum_{i\in\mathbb{C}^t} p_i\left(F_i(w^t) - F_i(w^*) + \frac{\mu}{2}\|w^t - w^*\|^2\right)\right. \\ & + \sum_{i\in\mathbb{S}^t} p_i\left(F_i(w^{t-k_i}) - F_i(w^*) + \frac{\mu}{2}\|w^{t-k_i} - w^*\|^2\right) \\ & + \sum_{i\in\mathbb{S}^t} p_i\langle w^t - w^{t-1}, \nabla F_i(w^{t-k_i})\rangle \\ & + \sum_{i\in\mathbb{S}^t} p_i\langle w^{t-1} - w^{t-2}, \nabla F_i(w^{t-k_i})\rangle \\ & + \cdots \\ & \left. + \sum_{i\in\mathbb{S}^t} p_i\langle w^{t-k_i+1} - w^{t-k_i}, \nabla F_i(w^{t-k_i})\rangle\right), \end{aligned} \tag{32}$$

where $\alpha = \frac{|\mathbb{C}^t|}{|\mathbb{C}^t|+|\mathbb{S}^t|}$ represents the proportion of the normal clients in the total clients, while $1-\alpha$ represents the proportion of stragglers.

Using the (28) in the above equation, we have:

$$\begin{aligned} B_2 \leq & -2\eta\sum_{i\in\mathbb{C}^t} p_i(F_i(w^t) - F_i(w^*)) - \eta\mu\alpha\|w^t - w^*\|^2 \\ & - 2\eta\sum_{i\in\mathbb{S}^t} p_i\left(F_i(w^{t-k_i}) - F_i(w^*) + \frac{\mu}{2}\|w^{t-k_i} - w^*\|^2\right) \\ & + (1-\alpha)K\eta^2E^2Q^2 + \eta^2K\sum_{i\in\mathbb{S}^t}\|\nabla F_i(w^{t-k_i})\|^2, \end{aligned} \tag{33}$$

where K is the hyperparameter of the VCM and there $K = max\{k_i\}$.

By the Assumption 1, we have:

$$\|\nabla F_i(w^t)\|^2 \leq 2L(F_i(w^t) - F_i^*). \tag{34}$$

Applying (34) to B_2, we obtain the following results:

$$\begin{aligned} B_2 \leq & -2\eta\sum_{i\in\mathbb{C}^t} p_i(F_i(w^t) - F_i^*) - \eta\mu\alpha\|w^t - w^*\|^2 \\ & - \eta\mu\sum_{i\in\mathbb{S}^t} p_i\|w^{t-k_i} - w^*\|^2 \\ & + (1-\alpha)K\eta^2E^2Q^2 + 2\eta^2KL\sum_{i\in\mathbb{S}^t} p_i(F_i(w^{t-k_i}) - F_i^*). \end{aligned} \tag{35}$$

Next, we analyze term C, which is defined as follows:

$$\begin{aligned} C = & 2\eta^2KL(1-\alpha)\Gamma_{\mathbb{S}} - 2\eta\alpha\Gamma_{\mathbb{C}} \\ & - 2\eta\sum_{i\in\mathbb{C}^t} p_i(F_i(w^t) - F^*) \\ & + 2\eta^2KL\sum_{i\in\mathbb{S}^t} p_i(F_i(w^{t-k_i}) - F^*), \end{aligned} \tag{36}$$

where $\Gamma_{\mathbb{S}} = \sum_{i\in\mathbb{S}^t} p_i(F^* - F_i^*)$ and $\Gamma_{\mathbb{C}} = \sum_{i\in\mathbb{C}^t} p_i(F^* - F_i^*)$. Γ represents the gap between the client-side optimal and the global optimal under non-IID settings.

Then, we can focus on analyzing the boundary of C_2:

$$\begin{aligned} C_2 = & -2\eta\sum_{i\in\mathbb{C}^t} p_i(F_i(w^t) - F^*) \\ & + 2\eta^2KL\sum_{i\in\mathbb{S}^t} p_i(F_i(w^{t-k_i}) - F^*) \\ = & 2\eta^2KL\sum_{i\in\mathbb{S}^t} p_i(F_i(w^{t-k_i}) - F_i(w^t) + F_i(w^t) - F^*) \\ & - 2\eta\sum_{i\in\mathbb{C}^t} p_i(F_i(w^t) - F^*) \\ \leq & 2\eta^2KL\sum_{i\in\mathbb{S}^t} p_i\left(\frac{1}{2\mu}\|\nabla F_i(w^{t-k_i})\|^2\right) \\ \leq & \frac{\eta^2KLE^2Q^2}{\mu}, \end{aligned} \tag{37}$$

where in the second equality, we use the $F_i(\bar{w}^t)$ to simplify the formula. In the first inequality, we use $F_i(w^t) - F^* \geq 0$ and the Assumption 3 to obtain the result. Then, we can combine the above results to obtain the following formula:

$$\begin{aligned} \mathbb{E}\|w^{t+1} - w^*\|^2 \leq & (1-\eta\mu\alpha)\mathbb{E}\|w^t - w^*\|^2 \\ & - \eta\mu\sum_{i\in\mathbb{S}^t} p_i\mathbb{E}\|w^{t-k_i} - w^*\|^2 + C. \end{aligned} \tag{38}$$

We define an auxiliary variable $V^t = \mathbb{E}\|w^t - w^*\|^2$. Based on (38), we obtain:

$$V^{t+1} \leq (1-\eta\mu\alpha)V^t - \eta\mu\sum_{i\in\mathbb{S}^t} p_iV^{t-k_i} + C. \tag{39}$$

To simplify the analysis, we introduce the Lyapunov function:

$$\Phi^t = V^t + \eta\mu\sum_{i\in\mathbb{S}^t} p_i\sum_{j=1}^{k_i} V^{t-j}. \tag{40}$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

According to the definition of the Lyapunov function, we have:

$$\Phi^{t+1} = V^{t+1} + \eta\mu \sum_{i\in\mathbb{S}^{t+1}} p_i \sum_{j=1}^{k_i} V^{t+1-j}. \tag{41}$$

Thus, we obtain:

$$\Phi^{t+1} \leq (1-\eta\mu\alpha)V^t - \eta\mu \sum_{i\in\mathbb{S}^t} p_i V^{t-k_i} + C + \eta\mu \sum_{i\in\mathbb{S}^{t+1}} p_i \sum_{j=1}^{k_i} V^{t+1-j}. \tag{42}$$

Noting that:

$$\begin{aligned}\sum_{i\in\mathbb{S}^{t+1}} p_i \sum_{j=1}^{k_i} V^{t+1-j} = &\sum_{i\in\mathbb{S}^t} p_i \sum_{j=1}^{k_i} V^{t-j} \\ &+ \sum_{i\in\mathbb{S}^{t+1}\setminus\mathbb{S}^t} p_i \sum_{j=1}^{k_i} V^{t+1-j} \\ &- \sum_{i\in\mathbb{S}^t\setminus\mathbb{S}^{t+1}} p_i \sum_{j=1}^{k_i} V^{t-j},\end{aligned} \tag{43}$$

and given that the difference between $\mathbb{S}^t$ and $\mathbb{S}^{t+1}$ is small, we approximate:

$$\sum_{i\in\mathbb{S}^{t+1}} p_i \sum_{j=1}^{k_i} V^{t+1-j} \approx \sum_{i\in\mathbb{S}^t} p_i \sum_{j=1}^{k_i} V^{t-j}. \tag{44}$$

Thus, we obtain:

$$\Phi^{t+1} \leq (1-\eta\mu\alpha)V^t + C. \tag{45}$$

Since $V^t \leq \Phi^t$, we can further derive:

$$\Phi^{t+1} \leq (1-\eta\mu\alpha)\Phi^t + C. \tag{46}$$

Consequently, we have:

$$\Phi^t \leq (1-\eta\mu\alpha)^t\Phi^0 + \frac{C}{\eta\mu\alpha}. \tag{47}$$

Given that $V^t \leq \Phi^t$, we conclude:

$$V^t \leq (1-\eta\mu\alpha)^t\Phi^0 + \frac{C}{\eta\mu\alpha}. \tag{48}$$

Therefore, $\mathbb{E}|w^t - w^*|^2$ converges to $\frac{C}{\eta\mu\alpha}$ at a linear rate. Specifically, there exist constants $D = \Phi^0$ and $\rho = 1-\eta\mu\alpha$ such that:

$$\mathbb{E}|w^t - w^*|^2 \leq D\rho^t + \frac{C}{\eta\mu\alpha}, \tag{49}$$

which implies that the algorithm converges to a neighborhood at a linear rate. □

VI. Experiments

In this section, we evaluate the performance, robustness, and scalability of DVAFL across three publicly accessible datasets: UCIHAR, CIFAR-10, and FashionMNIST, thereby illustrating the framework's efficacy in mitigating the straggler problem. Furthermore, we assess the generalizability of DVAFL through validation on supplementary datasets, including California Housing and Google Speech Commands.

A. Experimental Setup

1) Datasets: We conduct experiments on the following datasets:

- *UCIHAR [36]:* The UCI Human Activity Recognition dataset contains recordings from 30 subjects performing daily activities while carrying a smartphone equipped with inertial sensors. It comprises 10,299 instances, each described by 561 features, categorized into 6 different activity classes.
- *CIFAR-10 [37]:* CIFAR-10 is a dataset of natural images comprising 60,000 32x32 color images across 10 classes, with 6,000 images per class. The dataset is divided into 50,000 training images and 10,000 test images.
- *FashionMNIST [12]:* FashionMNIST consists of 70,000 grayscale images of Zalando's article items, each sized 28x28 pixels. The dataset includes 60,000 training examples and 10,000 test examples, with each image labeled into one of 10 classes.
- *California Housing [38]:* The California Housing dataset contains 20,640 instances, each with 8 features, representing various attributes of housing in California. The target variable is the median house value for California districts, with values ranging from $14,999 to $500,001.
- *Google Speech Commands [39]:* This dataset contains 105,829 audio clips of spoken commands, each lasting 1 second. The dataset includes 12 classes, with each class representing a different spoken command.

Among these datasets, california housing and Google speech commands are used to validate the generalizability of DVAFL. The comprehensive experimental results pertaining to this validation are presented in Section VI-B7.

In most of our experiments, the training dataset is partitioned across $N = 35$ devices. We consider two types of data distributions: (1) Independent and Identically Distributed (IID) and (2) non-IID. In the IID setting, the training data is randomly divided among the N devices. In the non-IID setting, the training data is distributed among the devices according to a Dirichlet distribution [40], which is controlled by the parameter β. A larger β results in a distribution that more closely resembles IID, while a smaller β leads to a more heterogeneous data distribution. In our experiments, we use $\beta = 0.1, 0.2, 0.5$ to simulate non-IID conditions.

2) Training Setting: We carry out all of our methods using the FedModule platform [41], running them on a 64-bit Ubuntu 21.04. We use processes to emulate clients in our experiments. Each client runs independently on a separate CPU core, with latency simulated using the sleep function. All model training is completed on the RTX4090. Client-server communication is handled via pipes.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

We trained Convolutional Neural Networks (CNNs) on the FashionMNIST and UCIHAR datasets and utilized the ResNet-18 [42] architecture for training on the CIFAR-10 dataset. Stochastic Gradient Descent (SGD) was used as the optimizer, with the learning rate set to 0.01. The CNN model used in our experiments consists of two convolutional layers, two pooling layers, and two fully connected layers. Each selected client performed local training for 2 epochs. The batch sizes were set to 64 for FashionMNIST, 64 for UCIHAR, and 128 for CIFAR-10. In FL, non-trainable parameters, such as the running mean and variance of Batch Normalization (BN) layers, are typically excluded from the aggregation process. This practice is consistent with previous studies [43], [44]. All experiments ensure that each algorithm has converged. For detailed settings, please see our code repository.[1]

3) Staleness Setting: In most of our experiments, we categorized clients into three types: 10 normal clients, 15 slow clients, and 5 very slow clients. We treated the very slow clients as stragglers, resulting in a straggler ratio of 15%. These three types of clients were used to simulate the different performance levels of real-world devices such as smartphones, smartwatches, and routers. The computational power of the normal clients was set to match that of the experimental machine, while the slow clients operated at 50% of the normal clients' capacity, and the very slow clients at 20%. We chose a 15% straggler ratio because it reflects a reasonable assumption in real-world edge computing scenarios. We also explored more complex scenarios involving client heterogeneity, which will be discussed in Section VI-B3.

4) DVAFL Framework Setting: In all of our experiments, we set $K = +\infty$, which means that the VCM does not discard any straggler updates and instead compensates for all of them. We also set the hyperparameter $\lambda = 0.9$ and $\gamma_{vcm} = 0.8$. We use the K-means clustering method for most of our experiments. For the CIFAR-10 and FashionMNIST datasets, we apply the second-moment estimation in (18), while the direct estimation in (14) is used for the UCIHAR dataset. In Section VI-B6, we use the version bias correction in (14) for all methods related to DVAFL.

5) Baseline Methods: We consider the baseline algorithm FedAvg [1] alongside the latest AFL algorithms, including TWAFL [21], EAFL [7], FedBuff [45], and FedAT [9]. TWAFL allows clients to upload only the shallow layers of the model during normal iterations, thereby improving convergence efficiency. EAFL groups clients based on their data distribution, using asynchronous communication within groups to accelerate convergence, while synchronous communication between groups ensures convergence accuracy. FedBuff uses a buffer to control the number of updates in each round. All these methods use the optimal hyperparameters as recommended in their respective papers.

B. Experiment Results

1) Performance Analysis: In this subsection, we compare the performance of DVAFL with the baseline methods FedAvg, TWAFL, EAFL, FedBuff, and FedAT, and examine how different datasets impact DVAFL.

During our experiments, after each training iteration, individual clients uploaded their model parameters to the server for aggregation. The aggregated global model was then tested on the designated test dataset, with test accuracy serving as the primary metric for performance evaluation. Higher test accuracy indicates better model performance.

We evaluate the performance of the methods based on the final time consumption and model accuracy, specifically the time required to reach a specified number of iterations and the accuracy achieved across multiple datasets. Fig. 5 presents the final results for model accuracy across six different methods, three datasets, and four data distribution scenarios. As shown in Fig. 5(a) and (b), although DVAFL required more training rounds than other methods, its overall training time was shorter, and its accuracy was higher. For instance, in Fig. 5(b), DVAFL-$\beta = 0.5$ achieved 90% accuracy in just 10min, while EAFL-$\beta = 0.5$ needed 13min to reach 88% accuracy, with no further improvement possible. This trend becomes even more pronounced as data heterogeneity increases. EAFL-$\beta = 0.1$ only reaches 80% accuracy, whereas DVAFL-$\beta = 0.1$ not only matches but even surpasses FedAvg-$\beta = 0.1$ in accuracy. In Fig. 5(c), while FedAT exhibits a higher convergence rate, DVAFL ultimately achieves superior model accuracy. This performance disparity is attributable to the inherent bias in FedAT's client grouping protocol, which systematically prioritizes updates from specific cohorts at the expense of others. Consequently, the maximum achievable accuracy of FedAT's global model is constrained. Conversely, DVAFL implements a compensation mechanism to ensure the comprehensive integration of all client contributions, including those from stragglers. By mitigating this systemic bias, the DVAFL framework enables the model to attain a higher level of performance.

These experiments demonstrate significant improvements in the efficiency of our method. By increasing the number of iterations, DVAFL not only achieves better model accuracy but also delivers faster performance, fulfilling the objectives of superior and efficient training.

To facilitate a straightforward comparison, we present the experimental data in a tabular format. Table II shows the time required by DVAFL and baseline methods to achieve different accuracy levels across three datasets under two distinct data distributions (IID and $\beta = 0.2$). On average, DVAFL achieved a speedup of 3–8× over FedAvg and was 4–5× faster than EAFL on the UCIHAR dataset. Similarly, for the FashionMNIST dataset, DVAFL was 5.8× faster than FedAvg and 2.5× faster than EAFL. For the CIFAR-10 dataset, DVAFL achieved a 2.1× speedup over FedAvg, while EAFL failed to reach the same level of accuracy.

2) Robustness Analysis in non-IID Settings: While our approach does not directly address the challenges posed by non-IID data, it effectively utilizes straggler updates, which helps prevent drift in the global model. This strategy enables the model to incorporate knowledge from all clients, thereby reducing the impact of non-IID data. To demonstrate this, we present detailed experimental results under non-IID conditions.

[1]https://github.com/NUAA-SmartSensing/DVAFL

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

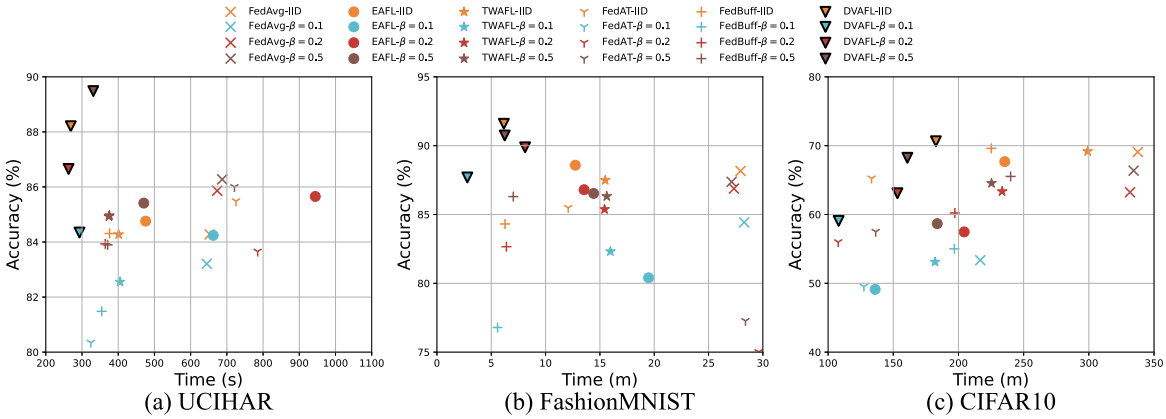
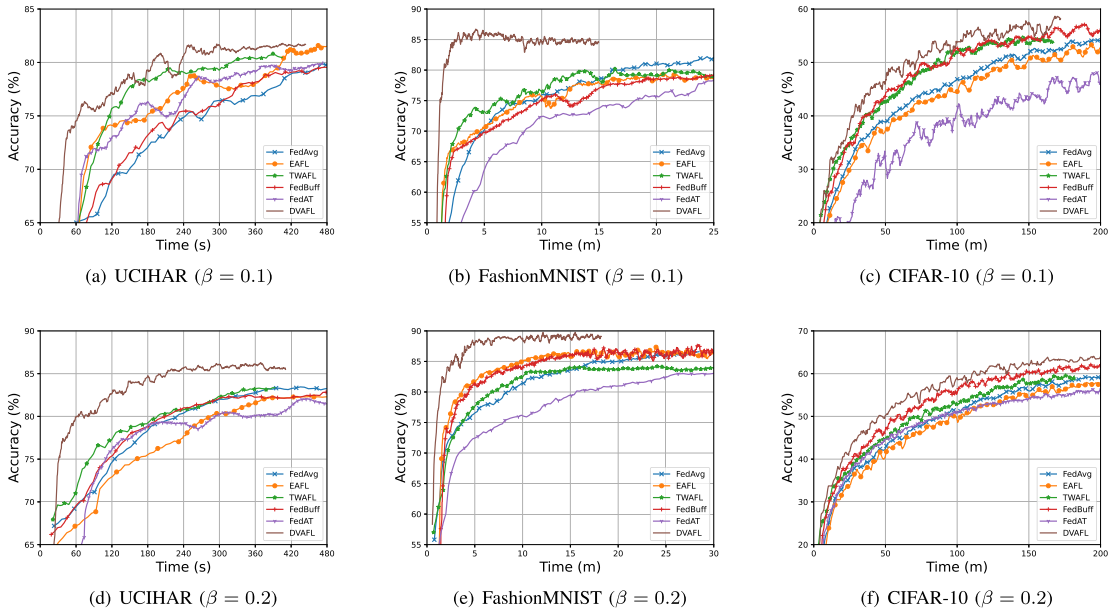


Fig. 5. Performance comparison between DVAFL and baseline methods over the running time in three different datasets. The notation [Method]-[Data Distribution] is used to represent the performance of each experiment. For example, DVAFL-IID represents the performance of DVAFL under IID distribution, and DVAFL-$\beta = 0.5$ represents the performance of DVAFL under a Dirichlet distribution with $\beta = 0.5$.

TABLE II
TRAINING TIME REQUIRED TO ACHIEVE TARGET ACCURACY ON THREE DATASETS

Methods	Time (s)									
	UCIHAR			FashionMNIST			CIFAR-10			
	IID	$\beta = 0.2$		IID	$\beta = 0.2$		IID	$\beta = 0.2$		
	84%	75%	83%	85%	70%	85%	60%	50%	55%	60%
FedAvg	639 (1×)	117	327 (1×)	896 (1×)	93	698 (1×)	10485 (1×)	5426	8036	13101 (1×)
EAFL	394 (1.6×)	127	409 (0.8×)	388 (2.3×)	87	306 (2.3×)	8632 (1.21×)	5867	9714	-
TWAFL	397 (1.6×)	59	276 (1.2×)	661 (1.4×)	73	520 (1.3×)	8251 (1.27×)	4271	7568	10413 (1.26×)
FedBuff	284 (2.25×)	40	293 (1.12×)	268 (3.34×)	**67**	361 (1.93×)	7006 (1.5×)	3406	5396	8496 (1.54×)
FedAT	320 (1.99×)	232	652 (0.5×)	-	845	-	10651 (0.98×)	5340	11433	-
DVAFL	**76 (8.4×)**	**26**	**101 (3.2×)**	**155 (5.8×)**	70	**85 (8.2×)**	**4816 (2.18×)**	**2707**	**4210**	**6498 (2.0×)**



(a) UCIHAR ($\beta = 0.1$) (b) FashionMNIST ($\beta = 0.1$) (c) CIFAR-10 ($\beta = 0.1$)

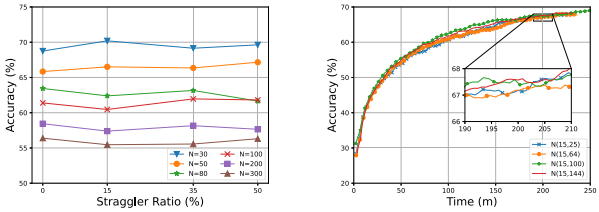
(d) UCIHAR ($\beta = 0.2$) (e) FashionMNIST ($\beta = 0.2$) (f) CIFAR-10 ($\beta = 0.2$)

Fig. 6. Performance comparison between DVAFL and baselines over the running time on three datasets in two different non-IID ($\beta = 0.1, 0.2$) scenarios.

Line graphs illustrate the accuracy over time for DVAFL and baseline algorithms under scenarios with $\beta = 0.1$ and $\beta = 0.2$. As shown in Fig. 6, DVAFL outperforms baseline algorithms in both convergence speed and accuracy in non-IID scenarios. For instance, in Fig. 6(a) and (d), DVAFL requires only 155 s and 50 s, respectively, to reach 80% accuracy, while FedAvg takes 442 s and 1185 s, respectively.

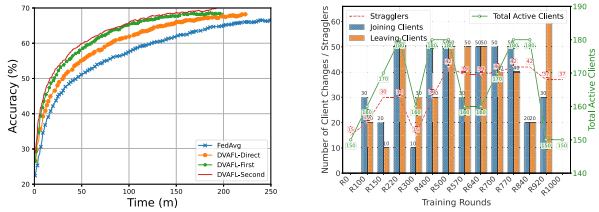
Although DVAFL exhibits greater model oscillation compared to other algorithms, this is due to more frequent model aggregation rounds within the same time frame, leading to more oscillations. Despite this, in Fig. 6(b) and (c), DVAFL achieves higher accuracy within the same runtime and even within the same number of iterations. This improvement in accuracy can be attributed to the fact that normal clients have high-quality data

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.



(a) Acc vs. Straggler Ratio (b) Acc vs. Staleness

Fig. 7. Performance of DVAFL under varying degrees of system heterogeneity on CIFAR-10.



(a) Performance Over Time (b) Dynamic Client Participation

Fig. 8. Scalability and adaptability of DVAFL in a large-scale dynamic federated learning scenario. We evaluate DVAFL with 600 clients on CIFAR-10, where client capabilities are heterogeneous and participation is dynamic—clients may join or drop out during training, and their computational performance may fluctuate over time.

distributions, whereas stragglers have less reliable data. In our approach, stragglers consistently aggregate their updates with a larger group of normal clients, which helps reduce the impact of data heterogeneity on the model's bias.

In the remaining Fig. 6(d) to (f), DVAFL achieves a final accuracy comparable to that of FedAvg while demonstrating a faster convergence rate than other methods. In our experiments, the EAFL method was observed to have stability issues, frequently converging to suboptimal solutions under certain conditions. This instability is attributed to EAFL's reliance on client grouping during training, as the determination of an optimal number of groups and their update frequency constitutes a significant challenge. Consequently, the performance of EAFL is highly sensitive to hyperparameter selection. In contrast, DVAFL employs fixed hyperparameters, which reduces the algorithm's dependency on extensive tuning. This inherent stability enhances its suitability for practical implementation by minimizing the need for laborious hyperparameter adjustments.

3) Performance Analysis in Heterogeneous and Dynamic Scenarios: In this section, we investigate the performance of our proposed DVAFL framework under various system conditions. We designed a series of experiments to assess its robustness in heterogeneous environments and its efficacy in large-scale, dynamic scenarios. These analyses highlight the contributions of the DWP and the VCM in ensuring stable and efficient model training.

To analyze the framework's robustness against system heterogeneity, we first conducted experiments under controlled conditions. Specifically, we evaluated DVAFL's performance by varying the total number of clients (30, 50, 80, 100, 200, 300) and the proportion of stragglers (0%, 15%, 35%, and 50%), as illustrated in Fig. 7(a). Critically, as the straggler ratio increases, DVAFL not only maintains its accuracy but, in some cases, surpasses the performance of the straggler-free system, peaking at a 35% straggler ratio. This highlights the effectiveness of the VCM in compensating for straggler updates and leveraging their data. We do observe a general decrease in accuracy as the number of clients increases, which is an expected consequence of heightened data heterogeneity from partitioning the dataset among more clients. These findings are consistent with our ablation analysis in Section VI-B4.

From a different perspective, inaccuracies in clients' local estimations of their task completion times can lead to misclassifications in straggler detection by the DWP, resulting in an overestimation of the straggler ratio. Nonetheless, DVAFL maintains a level of accuracy under a 50% straggler ratio that is comparable to that under 15%, indicating a high degree of robustness to potential prediction errors in local execution time estimation.

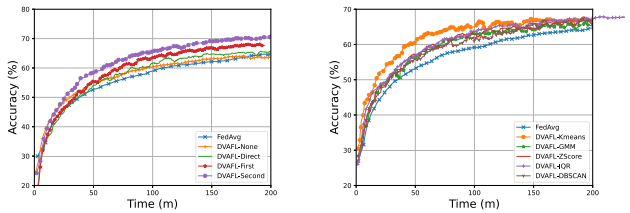
In a second set of experiments, we simulated more realistic latency conditions by modeling client training and communication times using different normal distributions: $N(15, 25)$, $N(15, 64)$, $N(15, 100)$, and $N(15, 144)$. As shown in Fig. 7(b), the accuracy difference between the best-performing case (DVAFL-$N(15, 25)$ at 68.56%) and the worst-performing case (DVAFL-$N(15, 144)$ at 67.37%) was a mere 1.19%. This demonstrates that DVAFL is highly robust to variations in client latency distributions, a quality attributable to the well-designed time window and the VCM's ability to mitigate the impact of delayed updates.

Building upon the analysis of static heterogeneity, we further evaluated DVAFL's efficacy in a more complex, large-scale, and dynamic environment. For this purpose, we conducted an experiment on the CIFAR-10 dataset with a total of 600 clients, simulating the continuous arrival and departure of participants and temporal variations in their performance. The dynamics of client participation are illustrated in Fig. 8(b). The CIFAR-10 dataset was partitioned into 150 sub-datasets using a Dirichlet distribution ($\beta = 0.5$), with the process replicated four times to ensure sufficient data per client. To simulate a varied environment, client performance was modeled using multiple distributions: 15% of clients were assigned fixed performance, 30% had their performance adjusted each round according to a normal distribution (N(20, 144)), and the remaining 55% had their performance randomly sampled (sampling range: $[10, 50]$) in every round.

As shown in Fig. 8(a), DVAFL achieved a final accuracy of 69.5%, outperforming FedAvg, which achieved 67.2%. Meanwhile DVAFL acheived around 2.5× speedup over FedAvg, which took 2000 seconds to reach the same accuracy. The results indicate that DVAFL effectively adapts to dynamic client participation and varying performance levels, maintaining high accuracy and stability throughout the training process.

Collectively, these experiments demonstrate that DVAFL maintains high accuracy and stability across diverse types of system heterogeneity and in large-scale dynamic settings. This robust performance is primarily due to the synergistic contributions of the DWP and VCM. DWP effectively manages stragglers by adjusting the time window, which reduces server waiting times and prevents the waste of computational resources. Concurrently, VCM efficiently compensates for and incorporates

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.



(a) VCM Methods (b) DWP Methods

Fig. 9. Performance comparison of different ablation methods of DVAFL under non-IID setting ($\beta = 0.5$) on CIFAR-10.

delayed updates from these stragglers into the global model, ensuring both stability and high convergence accuracy. These findings underscore our method's capacity to handle clients with varying latencies, thereby significantly improving overall system efficiency and robustness.

4) Ablation Analysis: In this subsection, we present ablation experiments to evaluate the effectiveness of the VCM and the DWP within the DVAFL framework. For the analysis of VCM, we compared the performance of DVAFL with and without VCM, as well as with various update correction strategies. Specifically, we examined four configurations: DVAFL-None, DVAFL-Direct, DVAFL-First, and DVAFL-Second. DVAFL-None denotes the baseline configuration in which VCM is not applied during training. In contrast, DVAFL-Direct, DVAFL-First, and DVAFL-Second incorporate VCM using version bias correction, first-order moment estimation, and second-order moment estimation, respectively, as defined in (14), (15), and (17). The influence of different clustering methods (namely K-means, GMM, Z-score, IQR, and DBSCAN) on classification efficacy was investigated. For a comprehensive evaluation, the experiment was conducted under the same dynamic client performance settings as detailed in Section VI-B3. This configuration involved assigning a fixed performance to 15% of the clients, adjusting the performance for 30% of clients each round based on a normal distribution N(20, 144), and randomly sampling the performance for the remaining 55% from the interval $[10, 50]$ in each round. The outcomes of these evaluations are summarized in Fig. 9.

As illustrated in Fig. 9(a), the DVAFL framework demonstrates notable improvements in training efficiency. In terms of final test accuracy, DVAFL-First and DVAFL-Second achieved accuracies of 67.8% and 70.2%, respectively, outperforming the baseline FedAvg method, which attained 66.47%. These gains can be attributed to the more effective utilization of straggler updates. Furthermore, all methods significantly accelerate training: DVAFL-Second reached a target accuracy of 65% in 82 minutes, compared to 107 minutes for DVAFL-First and 198 minutes for FedAvg. These findings confirm that the proposed update correction strategies within DVAFL facilitate faster global model convergence. While DVAFL-Direct and DVAFL-None yielded comparable final accuracies to FedAvg, they required only approximately 70% and 85% of the training time, respectively.

Fig. 9(b) presents a comparative analysis of the DVAFL framework's performance using different clustering strategies for the DWP. In this set of experiments, the version bias correction method was consistently employed for VCM. The results

TABLE III
EFFECT OF γ_{VCM} AND β PARAMETERS ON CIFAR-10 ACCURACY (%)

γ_{vcm}	Direct	First (β)			Second (β)		
		0.99	0.90	0.80	0.99	0.90	0.80
1.0	66.3	67.7	**67.8**	67.4	67.9	69.0	68.9
0.9	**66.7**	65.9	66.9	66.3	68.5	68.6	67.8
0.8	65.9	66.2	67.3	67.2	68.9	**70.2**	69.3

TABLE IV
EFFECT OF VCM-k PARAMETER ON CIFAR-10 ACCURACY (%)

VCM-k	**Methods**			**Memory**
	Direct	First	Second	
0	62.4	62.4	62.4	1×
2	63.8	65.4	66.5	3×
4	64.6	66.8	67.0	5×
8	65.7	67.2	69.1	9×
∞	66.1	67.8	**70.2**	∞

TABLE V
EFFECT OF DWP-γ_{DWP} ON ACCURACY AND TIME

γ_{dwp}	Accuracy	Time
0.2	70.2	**173min**
0.5	70.5	179min
0.8	**70.7**	186min

demonstrate that K-means, IQR, and Z-score-based clustering yield comparable final accuracies of approximately 67.6%, with the IQR method exhibiting a marginal performance advantage at 67.8%. In contrast, the GMM and DBSCAN clustering approaches resulted in lower accuracies of 65.2% and 65.4%, respectively. Furthermore, a significant distinction was observed in convergence speed. Owing to its minimal computational overhead, the K-means method converged approximately twice as fast (a 2× speedup) as the other clustering techniques. The other methods demonstrated comparable convergence rates, achieving an approximate 1.6× acceleration. These findings collectively indicate that the DWP component effectively mitigates the adverse impact of stragglers, thereby accelerating model convergence. The choice of clustering algorithm has a discernible impact on performance, with K-means and IQR emerging as the most effective options, offering a superior balance of accuracy and computational efficiency.

5) Parameter Sensitivity Analysis: This section presents a parameter sensitivity analysis to evaluate the impact of the hyperparameters γ_{vcm}, β, k, and γ_{dwp} on model accuracy and training time, utilizing the CIFAR-10 dataset. The empirical results for various parameter configurations are detailed in Tables III, IV, and V.

As shown in Table III, the analysis demonstrates that increasing γ_{vcm} improves model accuracy for both the Direct and First-order methods. This indicates that a higher contribution from the VCM aids these methods by potentially minimizing discrepancies between local and global models. However, the opposite trend is observed in the Second-order method, where smaller γ_{vcm} values result in better performance. This can be

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

explained by the stronger gradient modifications in second-order methods; a larger γ_{vcm} places greater emphasis on global model updates, which can reduce the sensitivity to local data and lead to a drop in accuracy. Thus, selecting an appropriate γ_{vcm} is method-dependent. For First-order methods, a moderate γ_{vcm} optimally balances local and global updates, leading to performance gains. In contrast, Second-order methods benefit from a lower γ_{vcm}, preventing global updates from overshadowing local data characteristics. Furthermore, the results indicate that for both first-order and second-order methods, the highest accuracy is achieved when $\beta = 0.90$. This highlights the critical role of the hyperparameter β in shaping model performance, particularly for first-order methods, where stronger regularization improves generalization.

Table IV details the impact of the VCM parameter k, the maximum staleness tolerance, on model accuracy. A key finding is the direct, positive correlation between the value of k and the performance of all evaluated VCM methods. The baseline performance is established at $k = 0$ (no staleness tolerance), where all methods yield an accuracy of 62.4%, a result identical to the FedAvg-drop approach that discards straggler updates. Increasing the staleness tolerance progressively enhances accuracy. At $k = 8$, the Direct, First, and Second-order methods achieve accuracies of 65.7%, 67.2%, and 69.1%, respectively. Performance peaks at the theoretical limit of $k = \infty$, reaching 66.1%, 67.8%, and 70.2%. The performance enhancement is achieved while carefully managing server-side resources. We acknowledge that storing $k + 1$ model versions introduces a memory requirement that scales linearly with k. However, this concern is substantially mitigated by the mechanism's operational efficiency. Since each VCM call involves only two global models, it is unnecessary to retain all versions in active memory simultaneously. By relegating inactive models to external storage, the practical memory overhead on the server is constrained to a modest twofold increase compared to the conventional FedAvg algorithm.

Table V evaluates the influence of the DWP parameter γ_{dwp} on model performance and computational efficiency. The results exhibit minimal variation, with accuracy figures ranging from 70.2% to 70.7% and training durations between 173 and 186 minutes. Lower values of γ_{dwp} are associated with faster convergence at the expense of slightly reduced accuracy, whereas higher values yield marginal accuracy gains but necessitate longer training times. These negligible performance differences suggest that the adaptive nature of γ_{dwp} effectively modulates the window size in response to real-time information, thereby ensuring both the stability of the model training.

In summary, the ideal value of γ_{vcm} is contingent upon the optimization method, with higher values being advantageous for the First-order method while lower values are preferable for the Second-order method to preserve local data integrity. Similarly, the meticulous tuning of β is revealed to be critical, underscoring its fundamental role in regularization. Furthermore, the analysis of the VCM parameter k uncovers a monotonic relationship between staleness tolerance and accuracy; however, this enhancement introduces a crucial trade-off, as the performance gains from increasing k are directly proportional to the requisite memory overhead. In contrast, the minimal impact of γ_{dwp} on final accuracy suggests its adaptive mechanism effectively ensures training stability without necessitating exhaustive tuning. Collectively, these results underscore the necessity of a deliberate and holistic approach to hyperparameter tuning, carefully balancing accuracy improvements against computational and memory constraints.

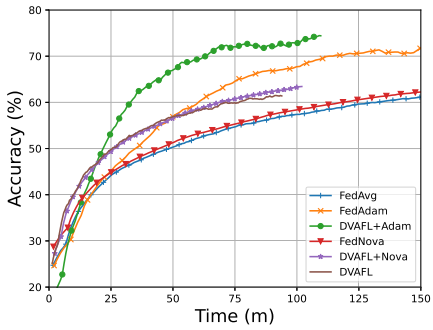
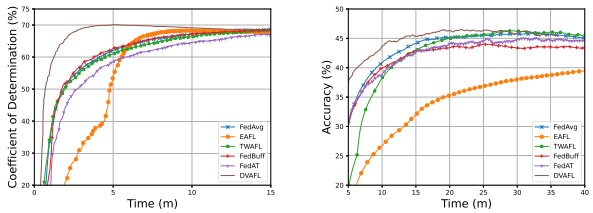


Fig. 10. Performance comparison between DVAFL and combinations of DVAFL with FedNova and FedAdam on CIFAR-10.

6) Framework Scalability Analysis: A key distinguishing feature of DVAFL is its non-intrusive nature, allowing it to integrate seamlessly with various methods, thereby improving overall accuracy and facilitating large-scale deployment. This section explores the synergies achieved by combining DVAFL with two synchronous FL algorithms, FedNova [46] and FedAdam [47], and assesses their impact on enhancing model performance. Our experiments, conducted on the CIFAR-10 dataset and illustrated in Fig. 10, reveal the substantial performance improvements introduced by DVAFL. As shown in Fig. 10, both FedNova and FedAdam outperformed FedAvg, with FedNova reaching 62% accuracy in 150min and FedAdam achieving 73% accuracy within the same time. When combined with these algorithms, DVAFL significantly reduced the time needed to reach the same accuracy levels. These results demonstrate that DVAFL can accelerate convergence while maintaining high accuracy. These findings underscore the critical role of DVAFL in enhancing convergence speed without sacrificing the accuracy delivered by advanced FL algorithms. The non-intrusive nature of DVAFL is especially important because it allows for this acceleration without requiring any modifications to existing FL frameworks. This reinforces DVAFL's value as a versatile and effective framework in AFL, providing efficiency gains without disrupting established algorithms.

7) Generalization to Diverse Datasets: To further assess the generalizability of our proposed method, we extended the validation of DVAFL to two additional and distinct tasks: regression on the California Housing dataset and audio classification on the Google Speech Commands dataset. The empirical results, illustrated in Fig. 11, confirm that DVAFL consistently outperforms the baseline methods. Specifically, DVAFL achieves a $3\times$ acceleration on California Housing and a $1.5\times$ acceleration on Google Speech Commands. Critically, these efficiency gains do not compromise model accuracy; on the California Housing dataset, DVAFL's R^2 score surpasses that of FedAvg by 1.8%. This robust performance is attributable to the dataset-agnostic

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.



(a) California Housing (b) Google Speech Commands

Fig. 11. Performance comparison of DVAFL and baseline methods on California Housing and Google Speech Commands datasets.

nature of the VCM method and the capability of DWP to dynamically establish a stable window, which ensures the accuracy and robustness of the aggregation process. In contrast, TWAFL, being an algorithm optimized for BN parameters, renders performance comparable to FedAvg, while EAFL exhibits a significant performance deficit on the Google Speech Commands dataset, failing to achieve a similar level of accuracy. This latter result suggests that EAFL's classification mechanism may be ill-suited for learning from audio data. The consistent outperformance of DVAFL across these fundamentally different data modalities and task types substantiates the robustness and broad applicability of our proposed framework.

VII. Conclusion

In this study, we introduce DVAFL, a novel AFL framework designed to improve model accuracy and real-time system performance by efficiently integrating straggler updates in scalable environments. Moving forward, we plan to further improve the DVAFL framework by exploring more efficient and precise methods for both the DWP and VCM, such as client-side local completion time prediction in DWP and improved weight correction method in VCM. Additionally, we intend to evaluate the scalability and concurrency-handling capabilities of DVAFL through experiments on real-world edge devices, such as smartphones and Internet-of-Things (IoT) platforms. Future work will also focus on optimizing the memory overhead of the VCM to improve its practicality in resource-constrained environments.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[2] M. Zhi et al., "Knowledge-aware parameter coaching for communication-efficient personalized federated learning in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 321–337, Jan. 2025.

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.

[4] A. Hard et al., "Federated learning for mobile keyboard prediction," 2018, *arXiv:1811.03604*.

[5] K. Luo et al., "Efficient coordination of federated learning and inference offloading at the edge: A proactive optimization paradigm," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 407–421, Jan. 2025.

[6] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

[7] Y. Zhou, X. Pang, Z. Wang, J. Hu, P. Sun, and K. Ren, "Towards efficient asynchronous federated learning in heterogeneous edge environments," in *Proc. IEEE Conf. Comput. Commun.*, 2024, pp. 2448–2457.

[8] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," 2019, *arXiv:1903.03934*.

[9] Z. Chai, Y. Chen, A. Anwar, L. Zhao, Y. Cheng, and H. Rangwala, "FedAT: A high-performance and communication-efficient federated learning system with asynchronous tiers," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 1–17.

[10] W. Wu, L. He, W. Lin, R. Mao, C. Maple, and S. Jarvis, "SAFA: A semi-asynchronous protocol for fast federated learning with low overhead," *IEEE Trans. Comput.*, vol. 70, no. 5, pp. 655–668, May 2021.

[11] H. Sun, Z. Gui, S. Guo, Q. Qi, J. Wang, and J. Liao, "GSSP: Eliminating stragglers through grouping synchronous for distributed deep learning in heterogeneous cluster," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2637–2648, Fourth Quarter, 2022.

[12] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[13] Q. Ma, Y. Xu, H. Xu, Z. Jiang, L. Huang, and H. Huang, "FedSA: A semi-asynchronous federated learning mechanism in heterogeneous edge computing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3654–3672, Dec. 2021.

[14] S. Chacon and B. Straub, *Pro Git: Everything you Need to Know About Git*. New York, NY, USA: Apress, 2014.

[15] Y. Zhou, Y. Shi, H. Zhou, J. Wang, L. Fu, and Y. Yang, "Toward scalable wireless federated learning: Challenges and solutions," *IEEE Internet of Things Mag.*, vol. 6, no. 4, pp. 10–16, Dec. 2023.

[16] A. Pantelopoulos and N. G. Bourbakis, "A survey on wearable sensor-based systems for health monitoring and prognosis," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 1, pp. 1–12, Jan. 2010.

[17] W. Dai, Y. Zhou, N. Dong, H. Zhang, and E. P. Xing, "Toward understanding the impact of staleness in distributed machine learning," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1153–1172.

[18] L. You, S. Liu, Y. Chang, and C. Yuen, "A triple-step asynchronous federated learning mechanism for client activation, interaction optimization, and aggregation enhancement," *IEEE Internet of Things J.*, vol. 9, no. 23, pp. 24199–24211, Dec. 2022.

[19] Q. Wang, Q. Yang, S. He, Z. Shi, and J. Chen, "AsyncFedED: Asynchronous federated learning with euclidean distance based adaptive weight aggregation," 2022, *arXiv:2205.13797*.

[20] L. Li et al., "FedSAE: A novel self-adaptive federated learning framework in heterogeneous systems," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–10.

[21] Y. Chen, X. Sun, and Y. Jin, "Communication-efficient federated deep learning with layerwise asynchronous model update and temporally weighted aggregation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 10, pp. 4229–4238, Oct. 2020.

[22] Y. Zhang et al., "FedMDS: An efficient model discrepancy-aware semi-asynchronous clustered federated learning framework," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 1007–1019, Mar. 2023.

[23] J. Sun et al., "FedSEA: A semi-asynchronous federated learning framework for extremely heterogeneous devices," in *Proc. 20th ACM Conf. Embedded Netw. Sensor Syst.*, 2022, pp. 106–119.

[24] G. Shi, L. Li, J. Wang, W. Chen, K. Ye, and C. Xu, "HySync: Hybrid federated learning with effective synchronization," in *Proc. IEEE 22nd Int. Conf. High Perform. Comput. Commun. IEEE 18th Int. Conf. Smart City IEEE 6th Int. Conf. Data Sci. Syst.*, 2020, pp. 628–633.

[25] Z. Xu, F. Yu, J. Xiong, and X. Chen, "Helios: Heterogeneity-aware federated learning with dynamically balanced collaboration," in *Proc. 58th ACM/IEEE Des. Automat. Conf.*, 2021, pp. 997–1002.

[26] J. MacQueen et al., "Some methods for classification and analysis of multivariate observations," in *Proc. Berkeley Symp. Math. Statist. Probability*, 1967, pp. 281–297.

[27] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 226–231.

[28] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[29] G. W. Snedecor and W. G. Cochran, *Statistical Methods*, 8th ed. Ames, IA, USA: Iowa State Univ. Press, 1989.

[30] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. Roy. Statist. Soc. Ser. Methodol.*, vol. 39, no. 1, pp. 1–22, 1977.

[31] Z. Z. C. Chen and Y. Zhao, "Leave no one behind: Unleashing stragglers' potential for accurate and realtime asynchronous federated learning," in *Proc. IEEE/ACM 32nd Int. Symp. Qual. Serv.*, 2024, pp. 1–10.

[32] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.

[33] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with Moreau envelopes," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 21394–21405.
[34] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAVG on non-IID data," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 13431–13457.
[35] X. Tang, S. Guo, and J. Guo, "Personalized federated learning with contextualized generalization," 2021, *arXiv:2106.13044*.
[36] E. Bulbul, A. Cetin, and I. A. Dogru, "Human activity recognition using smartphones," in *Proc. 2nd Int. Symp. Multidisciplinary Stud. Innov. Technol.*, 2018, pp. 1–6.
[37] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Tech. Rep. TR-2009-006, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf
[38] R. K. Pace and R. Barry, "Sparse spatial autoregressions," *Statist. Probability Lett.*, vol. 33, no. 3, pp. 291–297, 1997.
[39] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, *arXiv:1804.03209*.
[40] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
[41] C. Chen, Z. Zhang, and Y. Zhao, "FedModule: A modular federated learning framework," 2024, *arXiv:2409.04849*.
[42] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[43] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," 2021, *arXiv:2102.07623*.
[44] Y. Wang, Q. Shi, and T.-H. Chang, "Why batch normalization damage federated learning on non-IID data?," 2023, *arXiv:2301.02982*.
[45] J. Nguyen et al., "Federated learning with buffered asynchronous aggregation," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 3581–3607.
[46] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 638.
[47] S. Reddi et al., "Adaptive federated optimization," 2020, *arXiv:2003.00295*.



Chuyi Chen received the BS and MS degrees from the Nanjing University of Aeronautics and Astronautics, in 2022 and 2025, respectively. He is currently working toward the PhD Degree in computer science with Nanjing University. His research interests are mainly federated learning, system heterogeneity, and edge computing.



Yanchao Zhao received the BS and PhD degrees in computer science from Nanjing University, Nanjing, China, in 2007 and 2015, respectively. He is currently a professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing. In 2011, he was a visiting student with the Department of Computer and Information Sciences, Temple University, Philadelphia, Pennsylvania. His research interests include wireless networks, mobile computing, edge computing, and device-free sensing. He has published more than 60 papers in prestigious conferences and journals, such as the *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Dependable and Secure Computing*, IEEE INFOCOM, *International Journal of Human-Computer Studies*, *IEEE Transactions on Knowledge and Data Engineering*, *ACM Transactions on Sensor Networks*, and AAAI. He is the recipient of the Jiangsu Provincial Excellent Youth Fund. Currently, he serves the organizing committees of several high-profile conferences including IEEE INFOCOM, ICCCN, ICPADS, and MASS. He is also a reviewer for top-tier international journals, such as the *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Network Magazine*, *IEEE Communication Magazine*, and *IEEE Internet of Things Journal*. He is honored with the Jiangsu Provincial Science and Technology Award for his contributions to the field.



Zhe Zhang received the BS degree from the North China University of Water Resources and Electric Power, Zhengzhou, China, in 2020, and the MS degree from Heilongjiang University, Harbin, China, in 2023. He is currently working toward the PhD degree in computer science and technology with the Nanjing University of Aeronautics and Astronautics. His research interests include federated learning, UAV networks, and continual learning.



Wenzhong Li (Member, IEEE) received the BS and PhD degrees from Nanjing University, China, both in computer science. He was an Alexander von Humboldt scholar fellow with the University of Goettingen, Germany. He is now a full professor with the Department of Computer Science, Nanjing University. His research interests include distributed computing, data mining, mobile cloud computing, wireless networks, pervasive computing, and social networks. He has published more than 100 peer-review papers at international conferences and journals, which include INFOCOM, UBICOMP, IJCAI, ACM Multimedia, ICDCS, the *IEEE Communications Magazine*, *IEEE/ACM Transactions on Networking* (ToN), *IEEE Journal on Selected Areas in Communications* (JSAC), *IEEE Transactions on Parallel and Distributed Systems* (TPDS), *IEEE Transactions on Wireless Communications* (TWC), etc. He served as program co-chair of MobiArch 2013 and registration chair of ICNP 2013. He was the TPC member of several international conferences and the reviewer of many journals. He is the principle investigator of three fundings from NSFC, and the co-principle investigator of a China-Europe international research staff exchange program. He is a member the ACM and China Computer Federation (CCF). He was also the winner of the Best Paper Award of ICC 2009 and APNet 2018. He was featured on Elsevier's Most Cited Chinese Researchers in 2022-2023.



Jie Wu (Fellow, IEEE) is chief scientist with China Telecom and director with China Telecom Cloud Computing Research Institute. Before he joined China Telecom, he was Laura H. Carnell professor with Temple University and the director with the Center for Networked Computing (CNC). His current research interests include mobile computing and wireless networks, routing protocols, network trust and security, distributed algorithms, applied machine learning, and cloud computing. He serves on several editorial boards, including the *IEEE/ACM Transactions on Networking*. He is/was general chair/co-chair for IEEE IEEE IPDPS'23, ACM MobiHoc'23, and IEEE CCGrid'24 as well as program chair/co-chair for IEEE INFOCOM'11 and CCF CNCC'13. He was an IEEE Computer Society distinguished visitor, ACM distinguished speaker, and chair for the IEEE Technical Committee on Distributed Processing (TCDP). He is a fellow of the AAAS. He is the recipient of the 2011 China Computer Federation (CCF) Overseas Outstanding Achievement Award. He is a member of the Academia Europaea (MAE).

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 29,2026 at 05:43:30 UTC from IEEE Xplore. Restrictions apply.